

Kaizen Infinities — Unified CRM

PHASE 1 → IMPLEMENTATION: TECHNICAL BRIDGE DOCUMENTATION

This package translates the Phase 1 wireframe and information architecture into the technical specification a development team needs to build it: data model, database schema, API contract, auth/RBAC, event architecture, geospatial architecture, frontend routes and components, and a phased delivery plan. It does not redesign the Phase 1 UX. It assumes the wireframe (Global CRM, People, Organizations, Institutions, Institution Map, Institution 360, Activities, Leads, Opportunities, MoUs, Education CRM, Executive Dashboard, Relationships, Timelines, AI insights, Geographic intelligence) as fixed and builds underneath it.

Confidentiality: This document contains proprietary Kaizen Infinities product architecture. Internal use only.

PART I

Part I — Executive Technical Summary

Recommended core architecture: keep MongoDB as the primary operational database. Model relationships as a first-class collection queried through aggregation lookups — a **logical relationship graph**, not a physical graph database. Use MongoDB Atlas geospatial (2dsphere) indexing for the Institution Map, MongoDB Atlas Search for full-text/fuzzy search, and a lightweight daily-aggregation collection for analytics rather than a separate data warehouse. Defer a dedicated vector/AI layer to Phase 2, using MongoDB Atlas Vector Search when needed rather than migrating to Postgres+pgvector.

Why this instead of a rewrite: the team already operates MongoDB in production for the Education CRM; migrating to PostgreSQL would require a parallel schema-design effort, a data migration, and a change in developer tooling with no capability the CRM actually needs in Phase 1 (the relationship and geospatial requirements are both well served by MongoDB's document model plus its native geospatial and search features). Part II details the evaluation.

How to read this document

- Parts II–V define the data and storage architecture.
- Parts VI–VIII define the API and frontend contracts developers build against.
- Parts IX–XVII define cross-cutting systems: auth, activities, geospatial, MoUs, education, notifications, search, files, audit, integrations, analytics, AI readiness, security, performance.
- Parts XIX–XXII turn all of the above into a phased build plan, a backlog, acceptance criteria, and an explicit list of what is decided vs. still open.

- Every ambiguity found in the wireframe is called out inline as an **implementation decision** box, with a recommended minimum resolution — never a silent assumption.

Part II — System Architecture

Layered architecture

Layer	Technology	Responsibility
Client	React SPA (existing frontend stack)	Renders Phase 1 screens; holds URL/local UI state; calls API layer only — no direct DB access.
API layer	Node.js REST API (existing backend stack)	Auth, validation, business rules, permission checks, orchestrates DB + search + cache + storage.
Primary datastore	MongoDB (Atlas)	System of record for all entities — people, organizations, institutions, relationships, activities, leads, opportunities, MoUs, education records.
Search	MongoDB Atlas Search (Lucene-based, same cluster)	Full-text and fuzzy search across people/orgs/institutions/leads/opportunities without a separate search cluster.
Geospatial	MongoDB native 2dsphere index	Institution Map viewport, radius, and district queries.
Cache	Redis (or Atlas in-memory equivalent)	District/institution counts, map cluster precomputation, session tokens, rate limiting.
File storage	Object storage (S3-compatible), metadata in MongoDB	MoU documents, contracts, attachments — binaries never stored in MongoDB documents.
Background jobs	Queue/worker (e.g. BullMQ on Redis, or Atlas Scheduled Triggers)	MoU expiry checks, notification dispatch, daily analytics aggregation.

Database evaluation

Option	Verdict	Reasoning
MongoDB (recommended)	Adopt / continue	Existing production investment and team familiarity; document model fits the mixed CRM + education domain (variable fields per institution/education record) without excess joins; native geospatial + search cover Phase 1 needs without extra infrastructure; relationships and timelines are modeled as documents, not requiring a separate engine.
PostgreSQL	Not recommended for Phase 1	Would give stronger relational integrity and mature SQL tooling, but requires a full schema migration and re-tooling with no Phase 1 requirement MongoDB can't meet. Reconsider only if transactional/financial integrity requirements grow substantially (see Part XVII).
PostgreSQL + pgvector	Not recommended for Phase 1	Solves an AI/embedding-search need the product doesn't have yet (Part XXX explicitly defers AI features). Revisit only if a heavy semantic-search or RAG feature is scoped.
Relational + graph hybrid	Not recommended	A physical graph database (e.g. Neo4j) adds an operational system to keep in sync with the primary store, for relationship depths (person↔org↔institution, 1–3 hops) that MongoDB aggregation \$graphLookup already answers. Reserve for a future feature that needs deep multi-hop graph traversal or graph algorithms (e.g. influence-path scoring) as a distinct, justified addition — not a Phase 1 default.

Implementation decision needed: MongoDB is retained as the single system of record. "Relationship" is implemented as a logical graph — a relationships collection traversed via aggregation (\$lookup / \$graphLookup) — not a physical graph database. This must not be revisited without a specific traversal requirement current tooling can't answer.

Migration & scaling implications

- No migration required for Phase 1 — continue on the existing MongoDB cluster/Atlas project.

- Scale vertically/by sharding on high-write collections (activities, audit_logs) if volume grows; shard key candidates: owner_id or tenant_id if multi-tenant is ever introduced (not required for Phase 1 — Kaizen is single-tenant).
- Search and geospatial both run on the same MongoDB cluster via Atlas features — no separate Elasticsearch cluster to operate for Phase 1 (Part XXIII revisits this if search volume/complexity outgrows Atlas Search).

Part III — Canonical Data Model

Entities fall into three groups. Universal CRM entities apply across every vertical Kaizen sells into. Education-specific entities extend the universal model for the Education CRM without duplicating it. System/security entities support auth and governance.

Universal CRM entities

Entity	Purpose	Key relationships	Owner field
Person	Canonical human record — one row per real human, regardless of how many roles they hold.	→ Relationship → Organization/Institution; → Activity; → Lead; → Opportunity	owner_id (CRM owner)
Organization	Any company, government body, vendor, or partner Kaizen deals with.	→ Institution (subtype); → Relationship; → Opportunity; → MoU	owner_id
Institution	Education-sector organization subtype (school/college/university/training body) carrying geospatial + academic fields.	extends Organization; → Person (contacts); → MoU; → Opportunity; → Student/Enrollment	owner_id
Relationship	First-class edge between any two entities with type, role, status, and time bounds.	from_entity ↔ to_entity (polymorphic)	owner_id
Lead	Early-stage interest record, pre-qualification.	→ Person; → Organization/Institution; → converts to → Opportunity	owner_id
Opportunity	Qualified, working deal.	→ Account (Organization/Institution); → Contact (Person); → Activity; → MoU; → Project	owner_id
Activity	Universal interaction record (call, meeting, visit, etc.).	related_* polymorphic references to Person/Organization/Institution/Lead/Opportunity/Project	actor_id
Task	Actionable follow-up, may originate from an Activity.	→ related entity; → assignee	assignee_id
Communication	Logged message content (email/WhatsApp thread) distinct from the Activity summarizing it.	→ Activity; → Person	actor_id

Entity	Purpose	Key relationships	Owner field
MoU	Formal agreement with an Organization/Institution.	→ Organization/Institution; → Document; → Activity; → Opportunity	owner_id
Campaign	Marketing/outreach initiative that generates Leads.	→ Lead	owner_id
Project	Delivery-phase work after an Opportunity is won.	→ Opportunity; → Activity; → Payment	owner_id
Payment	Financial transaction tied to an Opportunity/Project/Enrollment.	→ Opportunity/Project/Enrollment	owner_id
Document	File metadata attached to any entity.	polymorphic attached_to	uploaded_by
Note	Free-text note attached to any entity (lighter-weight than Activity).	polymorphic attached_to	author_id
Tag	Free-form label attachable to any entity for filtering.	polymorphic attached_to	—

Education-specific entities

Entity	Purpose	Key relationships
Course	Catalog offering (curriculum, duration, fees).	→ Enrollment; → CareerPath
CareerPath	Grouping of courses toward an outcome (e.g. job role).	→ Course
Student	Education-context profile layered on a Person (see Part V).	1:1 → Person; → Enrollment
Enrollment	A Student's registration in a Course/batch.	→ Student; → Course; → Payment; → Attendance
Attendance	Per-session attendance record.	→ Enrollment
DailyProgress	Per-day academic progress log.	→ Enrollment
Recommendation	System/counsellor-suggested next course or action for a Student.	→ Student; → Course

System / security entities

Entity	Purpose	Key relationships
User	Login identity for a Kaizen employee (distinct from Person, which represents anyone, including non-employees).	→ Role; may reference Person if the employee is also tracked as a contact
Role	Named permission set (Founder, Admin, Sales, etc.).	→ Permission
Permission	Resource + action grant.	→ Role
AuditLog	Immutable change record.	polymorphic subject reference

Every entity carries: `_id`, `created_at`, `created_by`, `updated_at`, `updated_by`, and a source field recording how the record entered the system (manual entry, import, integration, conversion). Soft-delete via `deleted_at` — hard deletes are reserved for governance-mandated erasure (Part XXIV).

Person Model

One Person record per real human. Roles (student, employee, decision maker, trainer, faculty, partner, referral source, candidate) are never separate Person records — they are Relationship or role-context records pointing at the same Person.

Structure

Person holds identity-only fields. Context is layered on through two mechanisms:

- **Relationship records** — e.g. `Person --studied_at--> Institution`, `Person --employed_by--> Organization`. These carry their own start/end dates so a person's history (studied here, then worked there) is preserved as multiple time-bounded edges, not overwritten fields.
- **Role-context profile records** where a role needs extra structured data beyond a relationship — e.g. `StudentProfile` (academic fields) or `TrainerProfile` (certifications) — each a 1:1 extension keyed by `person_id`, created only when that context is needed.

Person fields

Field	Required	Notes
_id	yes	Canonical identifier.
full_name	yes	
primary_phone	yes*	*At least one of phone/email required at creation for deduplication.
primary_email	yes*	
additional_phones[] / additional_emails[]	no	Captured over time from different activities/sources.
current_location {city, district, state}	no	
social_identifiers {linkedin, whatsapp}	no	Only where a workflow actually uses it (e.g. WhatsApp activity logging).
external_ids[] {system, id}	no	Links to source system records — education LMS ID, import batch ID, etc.
source	yes	manual import integration conversion referral.
created_by / updated_by / created_at / updated_at	yes	Audit baseline.
dedupe_status	yes	active merged flagged_duplicate.
merged_into	no	Set when this record was merged into a surviving Person.

Deduplication & identity resolution

- At creation, the API checks phone + email against existing Person records before insert; an exact match on either blocks silent duplicate creation and surfaces the existing record to the user ("this person may already exist").
- Fuzzy name+phone matches (e.g. same phone, different name spelling) surface as a merge suggestion in a review queue rather than auto-merging.
- Merge operation: one record becomes canonical, the other's relationships/activities are re-pointed to the canonical _id, and the losing record is kept with dedupe_status: merged and merged_into set — never hard-deleted, so historical references and audit trails stay valid.

Implementation decision needed: Exact match on phone or email is the required minimum dedup rule for Phase 1. Fuzzy matching (name-similarity, phonetic phone/name combinations) is valuable but is a product-tuning exercise, not a Day 1 requirement — flag it for Phase 2 unless the sales team reports duplicate volume that makes it urgent.

Organization / Institution Model

Institution is an Organization subtype, not a parallel entity: education-sector organizations carry the base Organization fields plus institution-specific fields (geospatial location, `institution_type`, `management_type`, academic identifiers). This is implemented as a single `organizations` collection with a `category` discriminator (`company` | `institution` | `government` | `vendor` | `partner` | `other`) and an `institution_details` sub-document present only when `category = institution` — avoiding duplicate contact/relationship/activity plumbing between two collections.

Field	Applies to	Notes
<code>name</code> , <code>category</code> , <code>industry</code>	all	<code>category</code> drives which sub-document is populated.
<code>parent_org_id</code>	all	Supports Parent → Subsidiary → Branch → Department hierarchy via self-reference; a Department is simply an Organization with a parent.
<code>institution_details.institution_type</code>	institution	<code>school</code> <code>college</code> <code>university</code> <code>training_institution</code> <code>other</code> .
<code>institution_details.management_type</code>	institution	<code>government</code> <code>private</code> <code>aided</code> <code>autonomous</code> .
<code>institution_details.location</code> { <code>lat</code> , <code>lng</code> , <code>address</code> , <code>district</code> , <code>taluk</code> }	institution	Backs the Institution Map — see Part XI.
<code>institution_details.external_identifier</code>	institution	Code from an authoritative education directory, where available — not fabricated (Part XI).
<code>contacts[]</code>	all	Denormalized quick-reference list of { <code>person_id</code> , <code>role_label</code> }; the authoritative link is still a Relationship record, not this array — see below.

Contact **roles** (Decision Maker, Influencer, Buyer, Placement Officer, Principal, Dean, HOD, HR, Technical Stakeholder) are stored as the `role` field on the Relationship connecting a Person to the Organization/Institution — never as separate Person records. One Person can hold multiple roles across multiple organizations simultaneously, each a distinct Relationship record.

Relationship Model

Relationship is a first-class, polymorphic collection — the mechanism behind every "this connects to that" in the product, including timelines, Institution 360, and the map's relationship-status filter.

Field	Notes
_id	Canonical identifier.
from_type / from_id	Polymorphic reference (Person, Organization, Institution).
to_type / to_id	Polymorphic reference.
relationship_type	studied_at employed_by decision_maker_for partner_of referred alumnus_of trainer_at vendor_of other.
role	Free-text/enum label used when relationship_type is a role-bearing org link (e.g. Placement Officer).
status	active inactive ended.
strength	weak moderate strong — manually set or derived from activity recency/frequency (Part XVI).
owner_id	CRM user accountable for the relationship.
start_date / end_date	Nullable end_date = ongoing.
source, notes, created_at, updated_at	Standard audit/provenance fields.

Temporal history is preserved by never overwriting a Relationship in place when its nature changes — a new role or employer produces a *new* Relationship record with its own start_date, and the prior one gets an end_date. A Person's full history (studied at Institution X 2018–2021, then employed by Company Y 2021–present) is the ordered set of their Relationship records, not a single mutable field.

Lead Model

Field	Notes
_id, person_id, organization_id, institution_id	Nullable except person_id — a Lead always points at a Person (create one if it doesn't exist yet).
source	website_form referral campaign walk_in event cold_outreach other.
vertical, product	Which Kaizen offering this Lead is for.
owner_id, stage, score, status	stage/status per the universal pipeline (see Opportunity Model); score is a simple 0–100 heuristic in Phase 1 (Part XXX).
created_at, last_activity_at, next_action	Drives the "lead untouched" notification (Part XVI).
converted_to_opportunity_id	Set on conversion.

Conversion creates (or reuses, via the dedup check in the Person Model) the Person/Organization records, then creates an Opportunity referencing them and stamps converted_to_opportunity_id on the Lead. The Lead record is kept, not deleted — it remains the pipeline-entry audit trail.

Opportunity Model

Universal macro-stage pipeline, with vertical-specific stages nested underneath rather than replacing it:

Universal stage	Meaning
Discovered	Identified as a potential account/contact.
Engaged	First contact made.
Qualified	Confirmed fit and authority.
Opportunity	Formally opened as a pipeline deal.
Proposed	Proposal/quote sent.
Negotiating	Terms under discussion.
Won / Lost	Closed.
Delivering	Won and in delivery (Project active).
Outcome	Delivery complete, outcome recorded.
Renew / Expand / Refer	Post-outcome pipeline re-entry.

A vertical's own stage list (e.g. Education admissions stages: Inquiry → Counselling → Application → Admission Offered → Enrolled) maps to one of these macro-stages via a maps_to_macro_stage lookup, so cross-vertical pipeline reporting (Part XV) stays possible without forcing every vertical into identical stage names.

Field	Notes
_id, account_ref (org/institution), contact_id (Person)	Core linkage.
vertical, product, owner_id	.
stage, vertical_stage, probability, value, currency	probability defaults from a stage-based lookup table, editable per deal.
expected_close_date, source	.
related_activities[], proposal_doc_id, contract_doc_id	References, not embeds.
outcome	won_delivered won_cancelled lost_reason enum.

Implementation decision needed: Lead scoring in Phase 1 is a transparent rule-based score (recency + activity count + source weight), not a model. Treat any request for a "smart" score as a Phase 2/AI-readiness item (Part XVI), not a Phase 1 build item.

Part IV — Database Schema Specification

MongoDB collections, one per canonical entity group. Embed only tightly-coupled, unbounded-append child data that is never queried independently (e.g. attendance rows within an enrollment would NOT embed — see note). Reference everything else by `_id`.

Collection	Key fields (beyond <code>_id</code> /audit)	Embed vs. reference
users	email, password_hash, role_id, status, last_login_at	References role_id → roles.
roles / permissions	name, permissions[] {resource, action}	permissions embedded in role — small, static, edited together.
persons	full_name, primary_phone, primary_email, additional_*, current_location, external_ids[], dedupe_status	See Part V. additional_phones/emails embedded (small, bounded).
organizations	name, category, parent_org_id, institution_details{}, contacts[]	institution_details embedded (1:1, always fetched with the org). contacts[] is a denormalized cache, not authoritative.
relationships	from_type, from_id, to_type, to_id, relationship_type, role, status, strength, owner_id, start_date, end_date	Referenced collection — high cardinality, queried independently by either side.
leads	person_id, organization_id, institution_id, source, vertical, owner_id, stage, score, status, converted_to_opportunity_id	Referenced.
opportunities	account_type, account_id, contact_id, vertical, stage, vertical_stage, value, probability, owner_id	Referenced. related_activities kept as an array of activity_ids for fast timeline pull, but activities remain the source of truth.
activities	type, actor_id, participants[], related_person_id, related_organization_id, related_institution_id, related_lead_id, related_opportunity_id, related_project_id, timestamp, duration, outcome, notes, attachments[], next_action, owner_id	Referenced — this is the polymorphic timeline source (Part X).
tasks	related_activity_id, related_entity_type/id, assignee_id, due_date, status, priority	Referenced.

Collection	Key fields (beyond _id/audit)	Embed vs. reference
mous	organization_id, institution_id, scope, vertical, owner_id, document_id, start_date, end_date, status	Referenced.
documents	filename, storage_key, mime_type, size, attached_to_type/id, version, uploaded_by	Referenced; binary lives in object storage, not MongoDB (Part XX).
notes / tags	body / label, attached_to_type/id, author_id	Referenced.
courses / career_paths	name, duration, fees, curriculum_ref, career_path_id	Referenced.
students	person_id (1:1), academic_details{}	Referenced to persons; see Part XIV.
enrollments	student_id, course_id, batch, status, enrolled_at	Referenced.
attendance	enrollment_id, session_date, status	Referenced, NOT embedded in enrollment — unbounded growth per enrollment would blow past MongoDB's per-document practical size limits and make the enrollment document expensive to load for a field almost never needed with it.
daily_progress	enrollment_id, date, notes, score	Referenced, same reasoning as attendance.
recommendations	student_id, course_id, reason, status	Referenced.
projects / payments	opportunity_id/project_id, amount, status, due_date	Referenced.
audit_logs	subject_type, subject_id, actor_id, action, before{}, after{}, timestamp, source, ip	Referenced, append-only, never updated.

Database indexing

Collection.field(s)	Index type	Purpose
persons.primary_phone, persons.primary_email	unique-ish (sparse) B-tree	Dedup lookup at creation (Part V).
persons.external_ids.id	B-tree	Integration lookups.
organizations.institution_details.location	2dsphere	Map viewport/radius queries (Part XI).
organizations.institution_details.district, institution_type, management_type	B-tree (compound with category)	Map filter panel.
relationships.{from_type,from_id}, {to_type,to_id}	compound B-tree	Timeline/360 lookups from either side of a relationship.
relationships.relationship_type, status	B-tree	MoU/relationship-status map filter.
activities.related_person_id / related_organization_id / related_institution_id / related_opportunity_id	B-tree, one per related_* field	Timeline queries (Part X).
activities.timestamp	B-tree, descending	Recency sort, "activity recency" map filter.
leads.stage, owner_id	compound B-tree	Pipeline board queries.
opportunities.stage, owner_id, expected_close_date	compound B-tree	Pipeline board and forecasting.
mous.end_date, status	compound B-tree	Expiry job (Part XII).
Atlas Search index: persons, organizations, leads, opportunities	Lucene text index	Global search (Part XXII).

Tradeoffs: every additional index speeds reads but costs write throughput and storage; the set above is deliberately limited to fields that back an actual filter/sort in the wireframe, not every field that could theoretically be queried. Compound indexes are ordered to match the most selective, most-filtered field first (e.g. district before institution_type, since district narrows the result set more).

Search Architecture

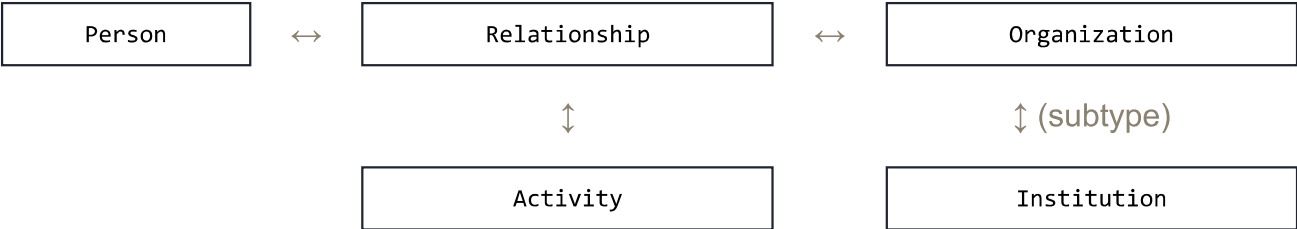
MongoDB Atlas Search (Lucene index on the same cluster) covers Phase 1 — a separate Elasticsearch/OpenSearch cluster is unjustified operational overhead for the query volume a Phase 1 CRM produces. Indexed fields: Person.full_name/phone/email, Organization/Institution.name, Lead/Opportunity identifying fields, MoU.scope, Course.name, Student (via Person). Supports exact match (phone/email/external_id via a term-level index), fuzzy name search (Lucene fuzzy matching), and prefix/autocomplete for GlobalSearch. Revisit Elasticsearch only if query complexity (e.g. relevance tuning across many entity types) or volume outgrows Atlas Search's limits.

File / Document Architecture

Files never live in MongoDB documents. Binaries go to S3-compatible object storage; MongoDB's documents collection holds only metadata (filename, storage_key, mime_type, size, attached_to_type/id, version, uploaded_by, created_at). Access control is enforced at the API layer (signed, time-limited download URLs issued only after a permission check on the parent entity — e.g. an MoU document requires mous:view) rather than making the bucket itself entity-aware. Versioning: each re-upload creates a new documents record with an incremented version and a shared document_group_id, rather than overwriting the storage object, so prior versions remain retrievable for audit.

Part V — Entity Relationship Diagram

Core cluster — the shape every other entity in the system hangs off:



Full relationship map:

From	Relationship	To	Cardinality
Person	studied_at	Institution	M:N via Relationship
Person	employed_by	Organization	M:N via Relationship
Person	decision_maker_for / holds role at	Organization / Institution	M:N via Relationship
Organization	partner_of	Kaizen (implicit root)	1:N
Institution	partner_of	Kaizen	1:N
Person	referred	Lead	1:N
Lead	converts_to	Opportunity	1:1 (on conversion)
Opportunity	account_ref	Organization / Institution	N:1
Opportunity	contact_id	Person	N:1
Activity	related_*	Person / Organization / Institution / Lead / Opportunity / Project	N:N (polymorphic, one activity can reference several)
MoU	organization_id / institution_id	Organization / Institution	N:1
Institution	has_many	Student (via Enrollment → Course delivered at Institution, where applicable)	1:N
Student	1:1	Person	1:1
Student	has_many	Enrollment	1:N
Enrollment	has_many	Attendance, DailyProgress	1:N
Document / Note / Tag	attached_to	any entity	N:1 polymorphic

Relationship is deliberately the only polymorphic hub between Person/Organization/Institution — Activity is a second, independent polymorphic hub for interaction history. Keeping these as two distinct collections (rather than one "everything is an edge" table) keeps each query shape simple: "who is this person connected to" vs. "what happened with this person" are answered by different indexes on different collections.

Part VI — API Specification

All routes are prefixed `/api/v1`, require a bearer session token unless noted, and return `{data, meta}` on success or `{error:{code,message}}` on failure. List endpoints share common query params: `page`, `limit`, `sort`, `order`; filter params are listed per-resource. 403 on a permission failure, 404 on missing resource, 422 on validation failure — used uniformly and not re-stated per row below.

Authentication

Method & Route	Purpose	Auth	Request / Params	Response
POST <code>/auth/login</code>	Email+password login	None	<code>{email, password}</code>	<code>{user, token, expires_at}</code>
POST <code>/auth/logout</code>	Invalidate session	Session	—	204
POST <code>/auth/refresh</code>	Rotate token before expiry	Refresh token	<code>{refresh_token}</code>	<code>{token, expires_at}</code>
POST <code>/auth/password-reset/request</code>	Send reset link	None	<code>{email}</code>	202
POST <code>/auth/password-reset/confirm</code>	Set new password	Reset token	<code>{token, new_password}</code>	204

Users

Method & Route	Purpose	Permission	Params/Body	Response
GET <code>/users</code>	List users	users:view (Admin)	<code>?role, ?status</code>	User[]
POST <code>/users</code>	Provision a user	users:create (Admin)	<code>{name, email, role_id}</code>	User (invite sent)
PATCH <code>/users/:id</code>	Edit / deactivate	users:edit	<code>{status, role_id, ...}</code>	User

People

Method & Route	Purpose	Permission	Params/Body	Response
GET /people	List/search people	people:view	?q, ?role_context, ?district, ?tag	Person[] + pagination
POST /people	Create person	people:create	Person fields (Part V) — server runs dedup check first, returns 409 with candidate matches if found	Person
GET /people/:id	Full profile incl. relationships summary	people:view	—	Person + relationships[] + recent activities[]
PATCH /people/:id	Edit	people:edit	partial Person fields	Person
POST /people/:id/merge	Merge duplicate	people:edit	{merge_into_id}	Person (canonical)
GET /people/:id/timeline	Activity timeline	people:view	?type, ?from, ?to, ?page	Activity[]

Organizations & Institutions

Method & Route	Purpose	Permission	Params/Body	Response
GET /organizations	List/search	organizations:view	?category, ?q, ?parent_org_id	Organization[]
POST /organizations	Create	organizations:create	Organization fields	Organization
GET /organizations/:id	Profile	organizations:view	—	Organization + relationships + activities summary
PATCH /organizations/:id	Edit	organizations:edit	partial fields	Organization
GET /institutions	Institution list (table view)	institutions:view	?district, ?institution_type, ?management_type, ?relationship_status, ?mou_status	Institution[] (category=institution)
GET /institutions/map	Map data — see Part XIV for full flow	institutions:view	?bbox (lat/lng bounds) OR ?lat,lng,radius; ?district, ?institution_type, ?management_type, ?relationship_status, ?mou_status, ?activity_recency, ?priority	{institution_id, name, type, lat, lng, district, relationship_status, mou_status, last_activity_at, priority}[] — deliberately thin; see Part XI
GET /institutions/:id/360	Full Institution 360 payload	institutions:view	—	See Part XIII data contract

Relationships

Method & Route	Purpose	Permission	Params/Body	Response
GET /relationships	List for an entity	relationships:view	?entity_type, ?entity_id (required — always scoped, never a global relationship list)	Relationship[]
POST /relationships	Create edge	relationships:create	{from_type, from_id, to_type, to_id, relationship_type, role, start_date}	Relationship
PATCH /relationships/:id	End/update (status, end_date, role)	relationships:edit	partial fields	Relationship

Leads & Opportunities

Method & Route	Purpose	Permission	Params/Body	Response
GET /leads	Pipeline list	leads:view	?stage, ?owner_id, ?source, ?vertical	Lead[]
POST /leads	Create	leads:create	Lead fields	Lead
POST /leads/:id/convert	Convert to Opportunity	leads:edit	{account_id?, contact_id?} (omit to reuse Lead's own person/org)	Opportunity + updated Lead
GET /opportunities	Pipeline board data	opportunities:view	?stage, ?owner_id, ?vertical	Opportunity[]
POST /opportunities	Create directly (non-lead origin)	opportunities:create	Opportunity fields	Opportunity
PATCH /opportunities/:id	Edit / change stage	opportunities:edit	partial fields — stage change emits OpportunityStageChanged (Part X)	Opportunity

Activities, Tasks, Communications

Method & Route	Purpose	Permission	Params/Body	Response
POST /activities	Log an activity against 1+ related entities	activities:create	{type, related_*_id(s), timestamp, duration, outcome, notes, next_action}	Activity — appears immediately in every related entity's timeline (Part X)
GET /activities	Cross-entity activity feed (e.g. "my activities today")	activities:view	?actor_id, ?type, ?from, ?to	Activity[]
GET /tasks	My/assigned tasks	tasks:view	?assignee_id, ?status, ?due_before	Task[]
POST /tasks	Create follow-up	tasks:create	{related_activity_id?, related_entity_type/id, assignee_id, due_date, priority}	Task
PATCH /tasks/:id	Complete/reassign	tasks:edit	partial fields	Task

MoUs

Method & Route	Purpose	Permission	Params/Body	Response
GET /mous	List	mous:view	?organization_id, ?institution_id, ?status, ?expiring_within_days	MoU[]
POST /mous	Create	mous:create	{organization_id/institution_id, scope, vertical, owner_id, start_date, end_date}	MoU (status=proposed)
PATCH /mous/:id	Advance lifecycle / renew	mous:edit	{status, end_date, ...}	MoU

Education: Courses, Students, Enrollments, Attendance, Progress

Method & Route	Purpose	Permission	Params/Body	Response
GET /courses, POST /courses	Catalog	education:view / education:create	—	Course[] / Course
GET /students, GET /students/:id	List / profile (wraps Person + academic_details)	education:view	?course_id, ?status	Student[] / Student+Person
POST /enrollments	Enroll a Student in a Course	education:create	{student_id, course_id, batch}	Enrollment
POST /attendance	Log a session's attendance	education:create	{enrollment_id, session_date, status}	Attendance
POST /daily-progress	Log a progress entry	education:create	{enrollment_id, date, notes, score}	DailyProgress
GET /recommendations	Suggested next actions for a Student	education:view	?student_id	Recommendation[]

Projects & Reports

Method & Route	Purpose	Permission	Params/Body	Response
GET /projects, PATCH /projects/:id	Delivery tracking	projects:view/edit	?opportunity_id	Project[] / Project
GET /reports/dashboard	Executive Dashboard KPI set	reports:view (visibility varies by role — Part IX)	?from, ?to	KPI summary object (Part XV)

Part VII — Frontend Route / Screen Specification

Route	Page	Required data	Permissions	Primary actions
/dashboard	Executive Dashboard	GET /reports/dashboard	reports:view (scope varies — Part IX)	Drill into KPI → filtered list view
/people, /people/:id	People list, Person profile	GET /people[, /:id], /:id/timeline	people:view	Create person, log activity, create relationship
/organizations, /organizations/:id	Organization list/profile	GET /organizations[, /:id]	organizations:view	Create org, add contact relationship
/institutions	Institution list (table)	GET /institutions	institutions:view	Filter, open Institution 360
/institutions/map	Institution Map	GET /institutions/map	institutions:view	Filter, cluster-zoom, select → drawer → 360 (Part XIV)
/institutions/:id	Institution 360	GET /institutions/:id/360	institutions:view	Log activity, create MoU, create opportunity
/leads, /leads/:id	Lead list/detail	GET /leads[, /:id]	leads:view	Convert to opportunity
/opportunities, /opportunities/:id	Pipeline board, Opportunity detail	GET /opportunities[, /:id]	opportunities:view	Change stage, log activity, create MoU/project
/activities, /tasks	Activity feed, task list	GET /activities, /tasks	activities:view, tasks:view	Complete task, log activity
/mous, /mous/:id	MoU list/detail	GET /mous[, /:id]	mous:view	Advance lifecycle, renew
/education/students, /education/students/:id	Student list/detail	GET /students[, /:id]	education:view	Enroll, log attendance/progress

Route structure follows the wireframe's own navigation groupings rather than inventing new ones; the one addition is separating `/institutions` (table) from `/institutions/map` as sibling views of the same filter state, since the wireframe shows both a list and a map for the same data.

Part VIII — Frontend Component Architecture

Component	Purpose	Key props	Data source	States
AppShell	Top-level layout: sidebar + header + content outlet	user, activeRoute	Session (global state)	authenticated / loading
Sidebar	Primary nav per RBAC-visible sections	navItems	Derived from user.role permissions	expanded/collapsed
GlobalSearch	Cross-entity search-as-you-type	onSelect	Search API (Part XXI)	idle/loading/results/empty
RecordHeader	Name, type badge, owner, quick actions for any record page	entity, actions[]	Server state (entity)	—
RecordTabs	Tab set for a record (Profile/Relationships/Activities/...)	tabs[], activeTab	URL state	—
DataTable	Sortable/paginated list (People, Organizations, Institutions, Leads, Opportunities)	columns, rows, sort, page	Server state + URL state (sort/page/filters)	loading/empty/error
FilterPanel / FilterDrawer	Facet filters (district, type, status, etc.)	facets, values, onChange	URL state	open/closed
KPI Card	Single dashboard metric with drill-through	label, value, delta, onClick	Server state (dashboard)	—
ActivityTimeline	Chronological activity list for any entity	entityType, entityId	GET /people organization institutions/:id/timeline	loading/empty
ActivityComposer	Log-activity form (opens from any record)	relatedEntity, onSave	Local form state → POST /activities	draft/submitting
RelationshipPanel	Relationship list + add-relationship affordance	entityType, entityId	GET /relationships	—
ContactCard	Person summary within an Organization/Institution's contact list	person, roleLabel	Server state (parent record)	—
OpportunityCard / PipelineBoard	Kanban card / board by stage	opportunities, onStageChange	Server state (opportunities)	dragging
StatusBadge / MoUBadge	Small colored status label	status	—	—

Component	Purpose	Key props	Data source	States
Map	Institution Map canvas — markers + clusters	institutions, filters, viewport	GET /institutions/map (viewport-scoped, Part XI)	loading/idle
MapMarker / MapCluster	Individual pin / cluster bubble	institution or clusterCount	Derived from Map's data	hover/selected
InstitutionDrawer	Slide-over summary before full 360	institutionId	GET /institutions/:id (lightweight) — see Part XIV	open/closed/loading
QuickCreate	Global "+" — create Person/Org/Lead/Activity from anywhere	entityType	Local form state → respective POST	—
Modal, Form, DatePicker, EntitySelector, FileUploader	Shared primitives	varies	—	—
NotificationCenter	Bell/inbox for events in Part XVI	notifications	GET /notifications (Part XVI)	unread count
TaskList	Assigned tasks widget	tasks, onComplete	GET /tasks	—
AllInsightCard	Read-only surfaced insight (Phase 2+ content, Part XVI)	insight	GET /insights (future)	—

State management

Data	Location	Why
User session, permissions	Global state	Needed by AppShell/Sidebar/every permission check, changes rarely.
Map filters, list filters, active tab, pagination, sort	URL state	Shareable/bookmarkable, back-button correct, and survives refresh — critical for the Map given how often users filter-then-zoom.
Entity records (Person, Institution, Opportunity, etc.)	Server state (cache with revalidation, e.g. React Query)	Source of truth is the API; avoids hand-rolled cache invalidation bugs after mutations (e.g. logging an activity must invalidate that entity's timeline).
Modal/drawer open state, form drafts, drag-in-progress	Local component state	Ephemeral, view-only, never needs to survive navigation.

Part IX — Authentication & RBAC Specification

Authentication (Phase 1 scope — deliberately minimal)

- Email + password login, bcrypt/argon2 hashed, JWT or opaque session token with server-side revocation list.
- Session expiry: 8-hour sliding token with refresh; forced re-login after 30 days idle.
- Password reset via emailed time-limited token (60 min expiry).
- User provisioning: Admin creates a User with a Role → invite email with a set-password link.
- User deactivation: soft — status=inactive blocks login and hides them from assignee pickers, but their historical owner_id/actor_id references remain intact for audit continuity.
- MFA: not included in Phase 1 — flagged as a Phase 2 item unless the org's security policy requires it Day 1 (ask, don't assume).

Roles

Founder/Executive, Admin, Sales, Telecaller, Education Counsellor, Trainer, Project Manager, Workforce/Placement, Finance, Marketing.

Permission matrix

V=View, C=Create, E=Edit, D=Delete, A=Assign, X=Export, F=Financial visibility. Blank = no access.

This is the Phase 1 default matrix; tune per resource as roles get real usage data.

Resource	Founder/Exec	Admin	Sales	Telecaller	Edu Counsellor	Trainer	Proj Mgr	Workforce/Placement	Finance	Marketing
People	VCEDA X	VCEDA X	VCEA	VCEA	VCEA	V	V	VCEA	V	V
Organizations	VCEDA X	VCEDA X	VCEA	V	V	V	V	VCEA	V	VC
Institutions	VCEDA X	VCEDA X	VCEA	V	VCEA	V	V	VCEA	V	V
Leads	VCEDA X	VCEDA X	VCEA	VCEA	VCEA			VCEA		VC
Opportunities	VCEDA XF	VCEDA X	VCEA	V	VCEA		V	VCEA	VF	V
MoUs	VCEDA X	VCEDA X	VCEA		VCEA		V	VCEA	V	V
Education (students/enrollment)	VCEDA X	VCEDA X	V	V	VCEA	VCE		VCEA	V	V
Projects	VCEDA X	VCEDA X	V			VCE	VCEA	V	VF	V
Payments	VCEDA XF	VCEDA XF					V	V	VCEDA XF	
Reports	VXF	VX	V	V	V	V	V	V	VXF	V

Implementation decision needed: Exact per-cell permissions above are a defensible Phase 1 default inferred from role names in the wireframe, not something the wireframe states explicitly. Confirm with the founder before locking Financial visibility (F) and Delete (D) grants in particular — those are the two columns with the highest business risk if wrong.

Part X — Activity / Event Architecture

Universal activity model

One `activities` collection, typed by `type` (call, whatsapp, email, meeting, visit, workshop, training, event, follow_up, proposal, payment, mou_activity, research, survey, other), carrying up to five optional `related*_id` foreign keys (person, organization, institution, lead, opportunity/project).

How one activity appears on multiple timelines without duplication

A timeline view (Person, Organization, Institution, Opportunity, Student) never stores its own activity list — it queries the single `activities` collection filtered on its own `related*_id` field, indexed per Part IV. Logging one visit that involved a Person (the Placement Officer) at an Institution, tied to an Opportunity, is a single activity document with all three `related_*` fields set — it then satisfies three different timeline queries from one write, with no fan-out or copy.

Domain events

Events are emitted for consumers that need to react asynchronously (notifications, audit, future analytics) — not a full event-sourcing architecture. The write to MongoDB is still the source of truth; events are a side-effect broadcast.

Event	Producer	Payload (key fields)	Consumers	Purpose
PersonCreated	POST /people	person_id, source	Audit log	Governance trail.
RelationshipCreated	POST /relationships	relationship_id, from, to, type	Institution 360 cache invalidation, Audit	Keep aggregated views fresh.
LeadCreated	POST /leads	lead_id, owner_id	Notification service	Notify owner of new assignment.
LeadConverted	POST /leads/:id/convert	lead_id, opportunity_id	Audit, Analytics daily rollup	Pipeline reporting.
OpportunityStageChanged	PATCH /opportunities/:id	opportunity_id, from_stage, to_stage	Notification service, Analytics rollup	Stage-change alerts, pipeline velocity.
ActivityCreated	POST /activities	activity_id, related*_ids, type	Timeline cache invalidation, Task auto-creation (if next_action set)	Keep timelines and follow-ups current.
MoUCreated / MouExpiring	POST /mous, scheduled job	mou_id, end_date, owner_id	Notification service	Renewal workflow (Part XII).
StudentEnrolled / StudentCompleted	POST /enrollments, PATCH status	student_id, course_id	Analytics rollup, Recommendations job	Progress and outcome tracking.
PaymentReceived	POST /payments (future)	payment_id, amount, related_id	Finance notification, Analytics rollup	Collections tracking.

Not every write emits an event — routine reads and minor edits (e.g. correcting a typo in a note) do not. Events are reserved for state transitions another part of the system genuinely needs to react to.

Notification / Task Architecture

Most rows below are event consumers from the table above, translated into a user-facing alert plus, where noted, an auto-created Task.

Trigger	Recipient	Channel	Priority	Action
Lead assigned	Assignee	In-app + email	Normal	Open Lead
Lead untouched (no activity in N days, configurable)	Assignee, then owner's manager after a further delay	In-app	High	Open Lead, log activity
Follow-up due (Task due_date reached)	Assignee	In-app + email	Normal	Complete Task
Opportunity stage changed	Owner, and Finance if stage=Won	In-app	Normal	Open Opportunity
MoU expiring (90/60/30/7 days)	MoU owner	In-app + email	Escalating with proximity to expiry	Renew or close out MoU
Task overdue	Assignee, manager after 2 days overdue	In-app	High	Complete/reassign Task
Payment pending	Finance, Opportunity owner	In-app	Normal	Follow up on payment
Student at risk (attendance/progress threshold)	Education Counsellor	In-app	High	Log intervention activity
Institution inactive (no activity in N days)	Institution owner	In-app	Low	Schedule outreach
Proposal pending (no response in N days)	Opportunity owner	In-app	Normal	Follow up

Notifications are stored (a lightweight notifications collection: recipient_id, type, payload, read_at) so NotificationCenter can list history, not just fire a toast. Email channel reuses the same event payload, not a separately maintained template flow.

Part XI — Geospatial / Institution Architecture

Scope confirmation

Tamil Nadu → District → Institution, defaulting the map view to Madurai district on load. Institution records carry lat/lng, address, district, taluk/block (where available), institution_type, management_type, and an external_identifier field for an authoritative directory code.

Implementation decision needed: The wireframe implies institution records exist at meaningful volume across Madurai district (and eventually Tamil Nadu). No unverified institution data should be seeded into the schema — populate district/institution directories from an authoritative source (e.g. state education department listings) as a data-loading task, not invented placeholder rows. Which authoritative source to license/scrape is a product decision, not a schema decision — the schema (Part IV) accepts any source once identified.

Map query architecture

Concern	Approach
What's queried	GET /institutions/map with either a viewport bbox or a center+radius, plus the active filter set (district, type, management_type, relationship_status, mou_status, activity_recency, priority). The query runs a single MongoDB \$geoWithin/\$nearSphere on the 2dsphere index, combined with the other filters in the same query — never fetched then filtered client-side.
What's indexed	2dsphere on institution_details.location (Part IV); compound B-tree on district + institution_type + management_type for the non-geo filters, since most map sessions filter by district before panning.
What's cached	Per-district institution counts and the default Madurai-district marker set are cached (Redis, 5–15 min TTL) since they're read far more often than institutions are edited.
What's calculated dynamically	relationship_status and mou_status shown per marker are computed at query time from the latest Relationship/MoU record for that institution — not stored redundantly on the institution document, so they can never drift out of sync.
What's precomputed	Marker clustering for zoomed-out views: a scheduled job buckets institutions into geohash cells and stores cell-level counts, so a zoomed-out viewport request returns cluster bubbles from a precomputed table instead of clustering thousands of points per request. Clicking into a cluster switches to the live per-marker query at that tighter viewport.

The map response is deliberately thin (institution_id, name, type, lat, lng, district, relationship_status, mou_status, last_activity_at, priority) — no contacts, activities, or documents. That data loads only once a marker is selected (Institution Drawer, then full 360), keeping the map payload small regardless of how much detail exists on any one institution.

Institution 360 — backend data contract

Section	Directly stored	Referenced (fetched on demand)	Aggregated / computed
Profile	organizations document (category=institution)	—	—
Contacts	—	persons via relationships where to_id=institution	Role label from relationship.role
Relationships	—	relationships collection	Relationship strength (Part V) computed from recent activity frequency
Activities / Timeline	—	activities where related_institution_id=this	Sorted, paginated by timestamp
MoUs	—	mous where institution_id=this	mou_status shown = latest MoU's status
Opportunities / Projects	—	opportunities/projects where account_id=this	Pipeline value sum (computed, not stored)
Programs / Students / Alumni	—	enrollments joined through students, where institution linkage exists via relationship (studied_at) or program delivery	Counts (enrolled, alumni) computed at request time
Documents	—	documents where attached_to=institution	—
AI insights	—	future insights service (Part XVI)	N/A in Phase 1 — placeholder section only

Map → Institution 360 interaction flow

Step	Frontend action	Backend/API	Data returned	Event	Permission check
1. Filter	User sets district/type/status filters	GET /institutions/map?... (URL state updates)	Thin marker list (see above)	—	institutions:view
2. Select institution	Click marker	GET /institutions/:id (lightweight profile, not full 360)	Name, type, address, relationship_status, mou_status, last_activity_at	—	institutions:view
3. Open drawer	InstitutionDrawer renders with #2's data	(no extra call — reuses #2)	—	—	—
4. Open full 360	User clicks "View full profile"	GET /institutions/:id/360	Full contract above	—	institutions:view
5. Load timeline (if not in initial payload)	ActivityTimeline tab mounts	GET /people institutions/:id/timeline?page=1	Paginated Activity[]	—	activities:view
6. Create activity	ActivityComposer submit	POST /activities {related_institution_id,...}	New Activity	ActivityCreated	activities:create
7. Update timeline	Timeline list re-fetches / optimistically prepends	(client cache invalidation)	—	—	—
8. Schedule next action	Composer's "next_action" creates a Task	POST /tasks {related_activity_id}	New Task	—	tasks:create

Part XII — MoU Architecture

MoU is first-class and supports multiple MoUs per Organization/Institution (a renewal creates a new MoU record referencing the prior one, rather than mutating it — preserving history).

Field	Notes
<code>_id</code> , <code>organization_id</code> / <code>institution_id</code>	At least one required.
<code>scope</code>	Free text / structured scope of the agreement.
<code>vertical</code> , <code>owner_id</code>	.
<code>document_id</code>	→ Document (Part XX) — the signed file.
<code>start_date</code> , <code>end_date</code>	.
<code>status</code>	proposed negotiating approved signed active expiring expired renewed.
<code>renewed_from_id</code>	Set on a renewal MoU, pointing at the MoU it supersedes.

Lifecycle & expiry automation

- Status transitions proposed → negotiating → approved → signed → active are user-driven (PATCH /mous/:id).
- A scheduled job runs daily, checking active MoUs' `end_date`: at 90/60/30/7 days out it sets `status=expiring` (idempotent) and emits `MoUExpiring`, which the Notification Architecture (Part XVI) turns into an alert to the owner.
- Past `end_date` with no renewal, the same job sets `status=expired`.
- Renewal creates a new MoU (`status=proposed`, `renewed_from_id` set) rather than editing the expired one — the expired MoU's own status becomes `renewed` once its successor is signed.

Part XIII — Education CRM Integration Architecture

The existing Education CRM's entities (students, enrollments, attendance, daily_progress, recommendations, courses, career_paths, leads) continue to function, re-based onto the universal Person model instead of standing alone.

Existing entity	Becomes
Student	Person (identity) + StudentProfile (1:1 academic_details sub-record) + one or more Enrollments — never a separate identity.
Lead (education)	A universal Lead with vertical=education; conversion creates/reuses the Person and, on enrollment, a StudentProfile.
Enrollment, Attendance, DailyProgress, Recommendation	Education-specific entities as defined in Part III, unchanged in behavior, now referencing student_id → a Person-backed Student rather than a standalone student table.
Course, CareerPath	Education-specific catalog entities, unchanged.

Concretely: a Student record is Person + StudentProfile, so the same human can simultaneously be an alumni (Relationship: studied_at, ended), an employee elsewhere (Relationship: employed_by, active), and a referral source (Relationship: referred → Lead) without three separate identities existing anywhere in the system — this is the mechanism validated end-to-end in Part XXI's critical test.

Implementation decision needed: Whether every historical education-only record (pre-migration) gets backfilled into Person/Relationship, or only new records go through the universal model going forward, is a migration-scope decision, not an architecture decision. Recommend backfilling at minimum current/active students and staff — full historical backfill can be a background migration job run after Phase 1E ships.

Part XIV — Integration Architecture

Integration	Direction	Data exchanged	Trigger	Auth	Failure handling	Phase
WhatsApp Business API	Bi-directional	Message content → Activity/Communication; outbound template sends	Webhook (inbound), API call (outbound)	API key/OAuth	Retry queue; failed sends surface as a Task	MVP
Telephony (click-to-call/call logging)	Bi-directional	Call metadata → Activity (duration, outcome)	Webhook on call end	API key	Missed webhook = manual activity log fallback	MVP
Email (SMTP/provider API)	Bi-directional	Thread content → Communication; logged as Activity	Send action, inbound webhook	OAuth/API key	Bounce handling → notify sender	MVP
Calendar (Google/Outlook)	Bi-directional	Meeting Activity ↔ calendar event	On Activity create/edit	OAuth	Sync conflict = last-write-wins with a warning	Phase 2
Payment gateway / UPI	Inbound	Payment confirmation → Payment record	Webhook on payment success	Signed webhook secret	Reconciliation job flags mismatches	MVP (Finance-critical)
LMS	Bi-directional	Attendance, progress, completion → DailyProgress/Enrollment	Scheduled sync or webhook	API key	Sync log + manual reconciliation view	Phase 2
HRMS	Inbound	Employee → User/Person provisioning	Scheduled sync	API key	Manual override always available	Future
ERP / Accounting	Bi-directional	Invoices, payments	Scheduled sync	API key	Reconciliation report	Future
Website forms	Inbound	Form submission → Lead	Webhook	Signed form token	Dedup check runs same as manual Lead creation	MVP
Marketing systems	Outbound	Lead/Contact export for campaigns	Scheduled or on-demand export	API key	—	Phase 2
Maps / geospatial (geocoding, tiles)	Outbound	Address → lat/lng at institution creation; map tile rendering	On institution create/edit; map render	API key	Manual lat/lng entry fallback if geocoding fails	MVP

MVP integrations are the ones the Phase 1 wireframe's activity logging and map cannot function acceptably without (WhatsApp/telephony/email logging, payments, website-form leads, geocoding). Everything else is real but sequenced later — building them now would delay the core CRM without a Phase 1 workflow depending on them.

Part XV — Analytics / Reporting Data Architecture

Metric group	Examples	Freshness
Pipeline	Revenue, pipeline value, conversion rate, stage velocity	Near-real-time (computed on dashboard load from live Opportunity data — volumes are low enough in Phase 1 that no pre-aggregation is needed)
Collections / Finance	Payments received, outstanding	Near-real-time
Education	Admissions, enrollment, attendance, completion, placement	Daily aggregated (attendance/progress volume makes per-request computation wasteful; a nightly job writes daily_metrics documents)
Geographic	Institution penetration by district, MoU penetration, opportunity density	Daily aggregated — these are exploratory/planning metrics, not operational ones.
Activity	Activity counts by type/owner	Near-real-time for "my activity today"; daily aggregated for team rollups.

A dedicated data warehouse is not justified for Phase 1 volumes. A single daily_metrics collection (one document per metric per day) written by a scheduled aggregation job covers every "daily aggregated" row above; the Executive Dashboard's near-real-time metrics query live collections directly with the indexes already defined in Part IV. Revisit a warehouse only if report complexity or query latency actually requires it.

Part XVI — AI Readiness Architecture

No AI agent, scoring model, or recommendation engine ships in Phase 1. The architecture instead makes sure the data these features will need is being captured correctly from Day 1 — retrofitting historical data later is far more expensive than logging it correctly now.

Future AI feature	Data it needs	Where Phase 1 already captures it
Lead scoring (model-based)	Historical lead outcomes, activity frequency/recency, source	leads + activities collections, already time-stamped and outcome-tagged
Opportunity probability	Stage history, time-in-stage, win/loss outcomes by vertical	opportunities.stage transitions via OpportunityStageChanged event log
Next-best-action	Activity sequences preceding a won/lost outcome	activities linked to opportunities, chronologically queryable
Course recommendation	Enrollment history, progress, outcomes	enrollments, daily_progress, recommendations (existing manual version)
Institution priority scoring	Relationship strength, activity recency, MoU status, opportunity density per institution	relationships.strength, activities.related_institution_id, mous, opportunities — all already queryable per institution
Cross-sell	Person's relationship graph across verticals	relationships + opportunities per person/organization
Churn / renewal risk	MoU expiry history, activity drop-off	mous.status history, activities recency
Candidate–job match	Student skills/course outcomes vs. employer opportunity requirements	enrollments outcomes + opportunities (workforce vertical)

When these features are scoped, MongoDB Atlas Vector Search can add embedding-based retrieval on top of the existing collections without a database migration — this is the concrete reason PostgreSQL+pgvector was not recommended in Part II.

Part XVII — Security, Audit & Data Governance

Security

- Encryption in transit (TLS everywhere) and at rest (Atlas-managed encryption at rest).
- Secrets (API keys, integration credentials) in a managed secrets store, never in source or plain config.
- PII (names, phone, email, address) and student data are the highest-sensitivity fields in this system — access to People/Students is gated by the RBAC matrix (Part IX), and export (X permission) is logged.
- Financial data (Payments, Opportunity value) visibility is a separate permission (F) from View, per Part IX's matrix, so Sales can work a deal without seeing margin/payment detail unless explicitly granted.
- Backups: daily automated Atlas backups with point-in-time recovery; retention per Kaizen's data-retention policy (a product decision, not specified in the wireframe).

Implementation decision needed: Data retention period and deletion-on-request policy (e.g. for a departed student/employee's PII) is a compliance/legal decision, not a technical one. The schema supports it (soft-delete plus an eventual hard-delete/anonymize job keyed on `deleted_at`), but the retention window itself needs sign-off before building the deletion job.

Audit log

Every write to a governance-relevant collection (Person, Organization, Institution, Relationship, Opportunity, MoU, Payment, Student, Enrollment) writes a corresponding `audit_logs` entry: `subject_type`, `subject_id`, `actor_id`, `action` (create/update/delete), `before/after` diffs, `timestamp`, `source`, and `IP/device` where available. `audit_logs` is append-only — no update or delete path exists in the API for it.

Part XVIII — Performance

Screen	Target	Approach
Executive Dashboard	< 1.5s load	Query live collections with the indexes in Part IV; add daily_metrics pre-aggregation (Part XV) if any single KPI query exceeds budget.
People / Organization search	< 500ms per keystroke-debounced query	Atlas Search index (Part XI/XVII), not a collection scan.
Institution list	< 1s	Compound index on district/type/management_type.
Institution Map	< 1s per viewport pan/zoom, even with thousands of institutions	Viewport-scoped 2dsphere query (never fetch-all-then-filter); precomputed geohash clusters at zoomed-out levels (Part XI); thin marker payload; client-side marker virtualization for the visible viewport only.
Institution 360	< 1.5s for the initial profile; tabs (timeline, contacts, MoUs) load independently and lazily	Contract in Part XI is split by tab specifically so the drawer/header renders before slower sections (timeline, students) finish loading.
Timeline	< 800ms per page of 20 activities	related_*_id indexes (Part IV), server-side pagination, never full timeline fetched at once.
Opportunity pipeline board	< 1s	Compound index on stage+owner_id; board loads one page per stage column, not the entire pipeline.

Part XIX — Development Phasing

Phase	Scope	Depends on
1A — Foundation	Auth, Users, Roles/Permissions, App Shell + Sidebar, MongoDB provisioning, Audit log plumbing	—
1B — Core CRM	Person, Organization, Relationship, Activity, Task, GlobalSearch (Atlas Search), Document upload	1A
1C — Commercial	Lead, Opportunity, pipeline board, conversion flow, Notifications for lead/opportunity events	1B
1D — Institution Network	Institution (Organization subtype), Institution Map, filters, geospatial indexing/clustering, Institution 360, MoU	1B (needs Relationship/Activity); 1C's Opportunity for the 360's opportunity tab
1E — Education Integration	StudentProfile on Person, Enrollment, Attendance, DailyProgress, Course/CareerPath, Recommendations, education-specific dashboard views	1B (Person/Relationship must exist first so Student is layered on, not rebuilt)
1F — Executive Dashboard & Reporting	KPI aggregation, daily_metrics job, dashboard screen	1C, 1D, 1E must all be producing real data first

This order follows data dependency, not the wireframe's navigation order: Institution (1D) depends on Relationship and Activity existing first (1B), and Education (1E) depends on Person existing first so students are never modeled as a standalone identity. The Executive Dashboard is sequenced last because it aggregates data every other phase produces — building it earlier would mean building it against fixtures, then rebuilding it against real data.

Part XX — Developer Backlog

Epic: Foundation (Phase 1A)

ID	Story	Dependencies	Priority	Relevant API/Entity	Type
F-1	Auth: login/logout/session/reset	—	P0	POST /auth/*, User	Full-stack
F-2	RBAC: role/permission model + middleware enforcement	F-1	P0	Role, Permission	Backend
F-3	App Shell + Sidebar with role-filtered nav	F-2	P0	—	Frontend
F-4	Audit log write-through on all governed collections	F-2	P1	AuditLog	Backend

Epic: Core CRM (Phase 1B)

ID	Story	Dependencies	Priority	Relevant API/Entity	Type
C-1	Person CRUD + dedup check on create	F-2	P0	Person, /people	Full-stack
C-2	Organization CRUD incl. institution category + hierarchy	F-2	P0	Organization, /organizations	Full-stack
C-3	Relationship CRUD + RelationshipPanel component	C-1, C-2	P0	Relationship, /relationships	Full-stack
C-4	Activity logging + polymorphic timeline query + ActivityTimeline/Composer	C-1, C-2	P0	Activity, /activities	Full-stack
C-5	Task creation from Activity next_action + TaskList	C-4	P1	Task, /tasks	Full-stack
C-6	GlobalSearch across Person/Organization via Atlas Search	C-1, C-2	P1	Search index	Full-stack
C-7	Document upload + attach-to-entity + FileUploader	C-1	P1	Document, object storage	Full-stack

Epic: Commercial (Phase 1C)

ID	Story	Dependencies	Priority	Relevant API/Entity	Type
S-1	Lead CRUD + list/filter	C-1, C-2	P0	Lead, /leads	Full-stack
S-2	Lead conversion flow (dedup-safe)	S-1	P0	/leads/:id/convert, Opportunity	Full-stack
S-3	Opportunity CRUD + PipelineBoard (drag-to-stage)	S-2	P0	Opportunity, /opportunities	Full-stack
S-4	Notification events for lead assignment / untouched / stage change	S-1, S-3, F-4	P1	Notification service	Backend

Epic: Institution Network (Phase 1D)

ID	Story	Dependencies	Priority	Relevant API/Entity	Type
I-1	Institution data model + 2dsphere index + district/type filters	C-2	P0	Institution fields, GET /institutions	Backend
I-2	Institution Map: viewport query, clustering job, Map/MapMarker/ MapCluster components	I-1	P0	GET /institutions/map	Full-stack
I-3	InstitutionDrawer + Institution 360 data contract endpoint	I-1, C-3, C-4	P0	GET /institutions/:id, /:id/360	Full-stack
I-4	MoU CRUD + lifecycle transitions + expiry scheduled job	C-2	P0	MoU, /mous	Full-stack
I-5	MoUExpiring notification wiring	I-4, S-4	P1	Notification service	Backend

Epic: Education Integration (Phase 1E)

ID	Story	Dependencies	Priority	Relevant API/Entity	Type
E-1	StudentProfile on Person + Enrollment CRUD	C-1	P0	Student, Enrollment, /students	Full-stack
E-2	Attendance + DailyProgress logging screens	E-1	P0	Attendance, DailyProgress	Full-stack
E-3	Course/CareerPath catalog CRUD	—	P1	Course, CareerPath	Full-stack
E-4	Recommendations list (manual, non-AI) tied to Student	E-1, E-3	P2	Recommendation	Full-stack

Epic: Executive Dashboard (Phase 1F)

ID	Story	Dependencies	Priority	Relevant API/Entity	Type
D-1	daily_metrics aggregation job	S-3, I-4, E-2	P1	daily_metrics collection	Backend
D-2	Dashboard screen + KPI Cards + drill-through	D-1	P1	GET /reports/dashboard	Full-stack

Part XXI — Acceptance Criteria

Critical journeys

1. **Create person** — submitting a new Person with a phone/email that doesn't match an existing record creates it; a match blocks creation and surfaces the existing record.
2. **Create organization** — Organization created with a category; institution category exposes `institution_details` fields.
3. **Create relationship** — a Relationship between any two entity types is creatable, appears on both entities' Relationship panels, and is time-bounded.
4. **Create lead** — Lead requires a Person; creating with a new person's details creates that Person first.
5. **Convert lead** — conversion produces exactly one Opportunity, no duplicate Person/Organization, and stamps the Lead as converted.
6. **Create opportunity** — Opportunity requires an account and contact; appears on the pipeline board in its stage's column.
7. **Create activity** — an Activity with multiple `related_*` ids appears on every one of those entities' timelines from a single write.
8. **View person timeline** — chronological, paginated, filterable by type.
9. **View organization timeline** — same, scoped to that organization.
10. **View institution map** — loads centered on Madurai district by default; markers render within budget (Part XVIII) even with a large institution count.
11. **Filter institutions by district** — map and list both narrow to the selected district using the same filter param.
12. **Filter institutions by relationship status** — computed from live Relationship data, not a stale stored field.
13. **Open institution from map** — marker click opens InstitutionDrawer with profile summary before the full 360 loads.
14. **View Institution 360** — every section in Part XI's data contract renders, each independently loading.
15. **View MoU** — MoU detail shows lifecycle status and linked document.
16. **Create MoU** — MoU created against an Institution/Organization starts at `status=proposed`.
17. **See institution activity timeline** — same mechanism as Person/Organization timelines, filtered to that institution.
18. **Create activity from institution** — ActivityComposer opened from Institution 360 pre-fills `related_institution_id`.

19. **Create follow-up task** — an Activity's next_action can produce a Task without re-entering context.
20. **Connect education student to person** — enrolling a Student reuses an existing Person if the phone/email matches; never creates a duplicate identity.
21. **Connect student to institution** — a studied_at Relationship links the Student's Person to the Institution.
22. **Connect alumni to employer** — ending the studied_at relationship and adding an employed_by relationship preserves both as history on the same Person.
23. **Connect employer to opportunity** — an Opportunity against that employer Organization references the same Person as contact where applicable.

Critical end-to-end test (validates the whole architecture)

This scenario must pass without producing a duplicate Person record at any step:

1. Student A is created as a Person with a studied_at Relationship to Institution X (College), which is located in Madurai district and appears correctly on the Institution Map with no current business relationship.
2. An employee discovers Institution X on the map (no relationship, so it shows as unengaged) and contacts its Placement Officer.
3. The Placement Officer is created as a Person with an employed_by (or role-bearing) Relationship to Institution X.
4. An Activity is logged for that contact, referencing both the employee (actor) and the Placement Officer (related_person) and Institution X (related_institution) — it appears on Institution X's timeline, the Placement Officer's timeline, and the employee's activity list, from the single write.
5. An MoU is created between Kaizen and Institution X; it appears in Institution X's 360 MoU tab.
6. An Opportunity is created against Institution X, linked to the Placement Officer (contact), the logged Activity, and the MoU.
7. Time passes: Student A graduates, becomes an alumnus (studied_at Relationship gets an end_date), and joins Company Y (a new employed_by Relationship is created on the same Person record).
8. Company Y becomes a new Opportunity, using the same Person (Student A) as a contact if relevant to that deal.
9. Verification: exactly one Person record exists for Student A and one for the Placement Officer, each carrying multiple time-bounded Relationships rather than multiple identities.

Part XXII — Open Decisions, Risks & Final Summary

Open decisions collected from this document

#	Decision	Where raised	Type
1	Fuzzy person deduplication (beyond exact phone/email match)	Part III — Person Model	Product decision (Phase 2 candidate)
2	Financial visibility (F) and Delete (D) grants per role	Part IX — RBAC	Product decision — needs founder sign-off before lock
3	Authoritative institution directory source for Tamil Nadu/Madurai data	Part XI — Geospatial	Product decision (data sourcing, not schema)
4	Historical education-record backfill scope into Person/Relationship	Part XIII — Education Architecture	Product/migration decision
5	Data retention window and deletion-on-request policy	Part XVII — Security/Governance	Legal/compliance decision
6	MFA requirement for Phase 1 login	Part IX — Auth	Product decision

Risks

- **Map performance at scale** — clustering/precomputation (Part XI, XVIII) must be built alongside the map, not bolted on after institution volume grows; retrofitting is expensive.
- **Permission matrix drift** — the Phase 1 RBAC matrix (Part IX) is a defensible default, not confirmed business rule; shipping it unconfirmed risks a Financial-visibility leak.
- **Duplicate identities** — the Person-centric model only holds if every entry point (manual creation, Lead conversion, Student enrollment, integrations) runs the same dedup check; any bypass reintroduces the duplicate-identity problem this architecture exists to prevent.

1. What is fully specified

Canonical data model and field-level entity definitions (Part III); MongoDB schema, collections, and indexes (Part IV); the ERD (Part V); the full API surface for every Phase 1 resource (Part VI); frontend routes and component architecture (Parts VII–VIII); the activity/timeline mechanism and domain events (Part X); the geospatial/map query architecture and Institution 360 contract (Part XI); MoU lifecycle and expiry automation (Part XII); the Person-centric education integration mechanism (Part XIII); phased build order and a concrete backlog (Parts XIX–XX); acceptance criteria including the end-to-end identity test (Part XXI).

2. What remains ambiguous

Exact institution data sourcing/volume for Tamil Nadu; exact fuzzy-match tolerance for deduplication; whether existing education data needs full historical backfill or only forward-going integration.

3. What requires a product decision

RBAC financial-visibility and delete grants (Part IX); institution directory sourcing (Part XI); backfill scope (Part XIII); MFA inclusion (Part IX).

4. What requires a technical decision

Choice of job scheduler for MoU-expiry/analytics jobs (Atlas Scheduled Triggers vs. an external worker); choice of object storage provider; whether Atlas Search suffices long-term or a dedicated search cluster is warranted once query volume is known.

5. What should NOT be built in Phase 1

- Any AI scoring/recommendation model (Part XVI) — only the data capture that enables one later.
- A physical graph database (Part II) — the logical relationship graph via aggregation is sufficient at Phase 1 relationship depth.
- A dedicated data warehouse (Part XV) — a daily aggregation collection covers Phase 1 analytics.
- MFA, LMS/HRMS/ERP integrations, marketing-system integrations, calendar sync (Part XIV) — all sequenced Phase 2/Future.

6. What developers can begin implementing immediately

Phase 1A (Foundation: auth, roles, app shell, audit plumbing) and the Person/Organization/Relationship/Activity core of Phase 1B (Part XIX–XX) have no open product decisions blocking them — implementation can start from this document today, in parallel with resolving the open decisions above.